

DG INTERNS HUB

Cybersecurity Internship Program

WEEK 4

BASIC NETWORK SETUP & INITIAL SECURITY TESTING

Prepared By: Prakhar Gupta

Role: Cybersecurity Intern

Submission Date: 30/Sept/2026

Table of Contents

1. Introduction to Network Security
2. Objective
3. Why Network Security Matters
4. Lab Setup Explanation
5. Network Diagram
6. Services Configuration
7. Nmap Scan Analysis
8. Wireshark Findings
9. Mini Project Output
10. Conclusion & Learning
11. Task Breakdown Summary
12. Challenges Faced
13. Bonus Work Considered
14. Conclusion

1. Introduction to Network Security

Network security is the practice of protecting the confidentiality, integrity and availability of data as it moves across a network. A core skill for any cybersecurity professional is being able to set up a small lab network, run services on it, and then use standard tools to observe and test that network from an attacker's perspective — exactly the workflow covered in this Week 4 assignment.

This report documents the setup of a small authorized lab network, the configuration and verification of a running web service, an Nmap scan of that service, and a Wireshark capture analysing the resulting traffic.

2. Objective

To understand basic network setup and perform initial security testing in a lab environment, covering: building a small multi-machine lab, running a network service, scanning it with Nmap, and analysing its traffic with Wireshark.

3. Why Network Security Matters

Every device on a network exposes some combination of open ports and running services, and each of these represents a potential entry point for an attacker. Before a system can be defended, its owner must first understand exactly what is running and reachable on it — this is the core purpose of the reconnaissance and testing workflow demonstrated in this report.

- Confidentiality: ensuring data is only accessible to authorised parties (e.g., not sent over unencrypted protocols like Telnet or plain HTTP).
- Integrity: ensuring data is not modified in transit without detection.
- Availability: ensuring services remain accessible to legitimate users, and are not disrupted by attacks or misconfiguration.

Scanning tools such as Nmap and traffic analysers such as Wireshark are two of the most fundamental tools used by both attackers and defenders to understand these properties of a live network — which is exactly why this assignment focuses on using them together.

4. Lab Setup Explanation

Component	Details
Hypervisor	VMware Workstation Pro
Attacker Machine	Kali Linux — IP 192.168.94.137
Target Machine	Metasploitable 2 (Ubuntu Linux) — IP 192.168.94.135
Network Mode	VMware NAT network, subnet 192.168.94.0/24
Web Service	Apache HTTP Server 2.2.8, running on the target (port 80)

Both machines are connected on the same isolated NAT network, allowing the attacker (Kali) machine to directly reach and test services on the target (Metasploitable) machine, while remaining fully isolated from any external or public network.

5. Network Diagram

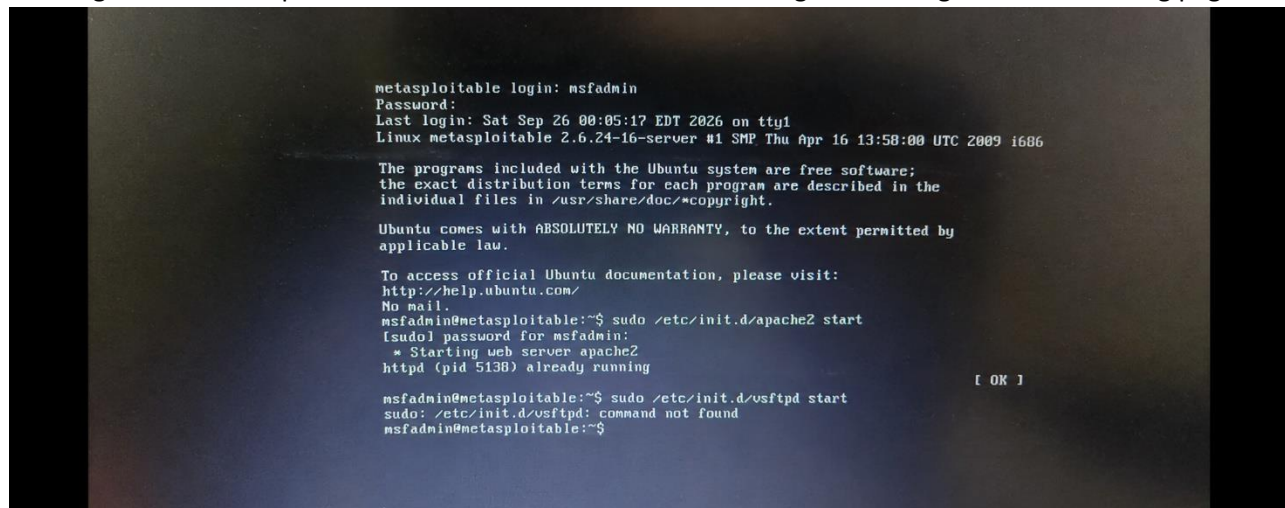
The lab topology is a simple two-node network sharing a single virtual switch:

[Kali Linux : 192.168.94.137] ↔ VMware NAT Switch (192.168.94.0/24) ↔ [Metasploitable 2 : 192.168.94.135]

Gateway: 192.168.94.2. Both machines communicate directly across the virtual switch; no traffic leaves the isolated lab network.

6. Services Configuration

The target machine's Apache web server was verified to be running and serving its default landing page.



```
metasploitable login: msfadmin
Password:
Last login: Sat Sep 26 00:05:17 EDT 2026 on tty1
Linux metasploitable 2.6.24-16-server #1 SMP Thu Apr 16 13:58:00 UTC 2009 i686

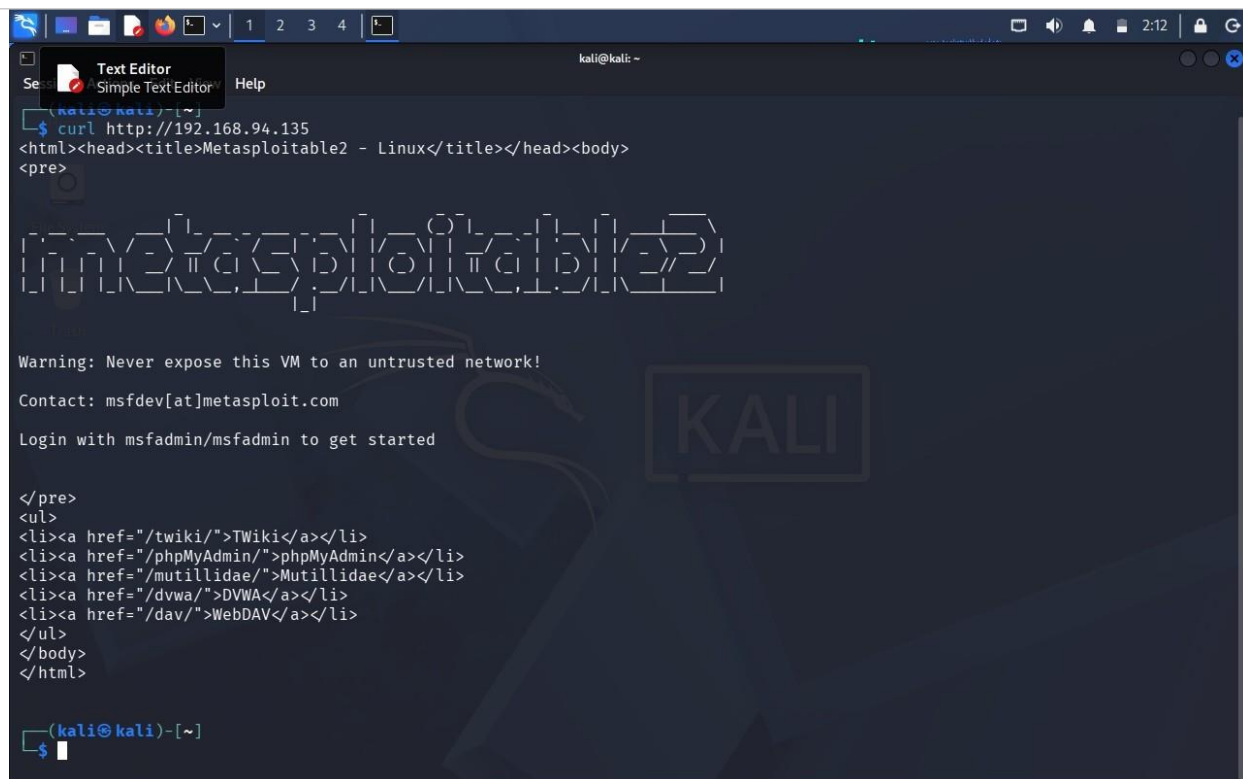
The programs included with the Ubuntu system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*copyright.

Ubuntu comes with ABSOLUTELY NO WARRANTY, to the extent permitted by
applicable law.

To access official Ubuntu documentation, please visit:
http://help.ubuntu.com/
No mail.
msfadmin@metasploitable:~$ sudo /etc/init.d/apache2 start
[sudo] password for msfadmin:
 * Starting web server apache2
httpd (pid 5138) already running

msfadmin@metasploitable:~$ sudo /etc/init.d/vsftpd start
sudo: /etc/init.d/vsftpd: command not found
msfadmin@metasploitable:~$
```

Metasploitable terminal: verifying the Apache web server (apache2) is running



```
(kali@kali)~$ curl http://192.168.94.135
<html><head><title>Metasploitable2 - Linux</title></head><body>
<pre>

Warning: Never expose this VM to an untrusted network!

Contact: msfdev[at]metasploit.com

Login with msfadmin/msfadmin to get started

</pre>
<ul>
<li><a href="/twiki/">TWiki</a></li>
<li><a href="/phpMyAdmin/">phpMyAdmin</a></li>
<li><a href="/mutillidae/">Mutillidae</a></li>
<li><a href="/dvwa/">DVWA</a></li>
<li><a href="/dav/">WebDAV</a></li>
</ul>
</body>
</html>

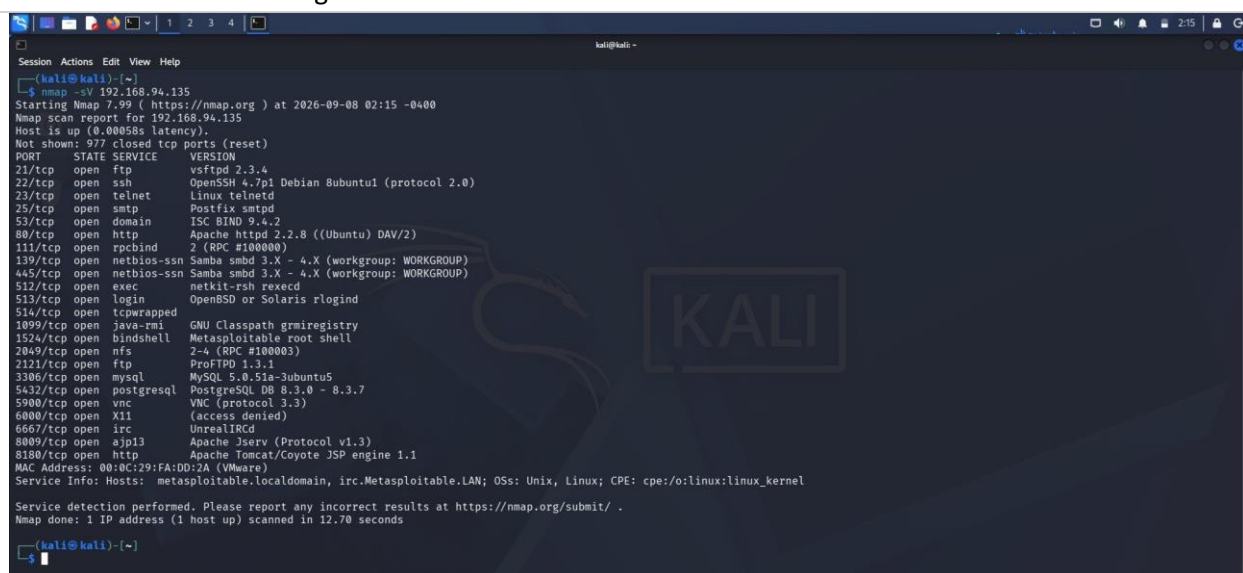
(kali@kali)~$
```

curl http://192.168.94.135 from Kali – confirms the web server responds with a valid HTML page

The response confirms the target's web server is reachable and serving content, including links to several bundled vulnerable web applications (TWiki, phpMyAdmin, Mutillidae, DVWA, WebDAV) that ship with Metasploitable 2 for security training purposes.

7. Nmap Scan Analysis

A service/version detection scan (nmap -sV) was run from Kali Linux against the target to identify all open ports and the exact software running on each.



```
(kali@kali)~$ nmap -sV 192.168.94.135
Starting Nmap 7.99 ( https://nmap.org ) at 2026-09-08 02:15 -0400
Nmap scan report for 192.168.94.135
Host is up (0.0005s latency).
Not shown: 977 closed tcp ports (reset)
PORT      STATE SERVICE      VERSION
21/tcp    open  ftp          vsftpd 2.3.4
22/tcp    open  ssh          OpenSSH 4.7p1 Debian 8ubuntu1 (protocol 2.0)
23/tcp    open  telnet       Linux telnetd
25/tcp    open  smtp         Postfix smtpd
53/tcp    open  domain       ISC BIND 9.4.2
80/tcp    open  http         Apache httpd 2.2.8 ((Ubuntu) DAV/2)
111/tcp   open  rpcbind      2 (RPC #100000)
139/tcp   open  netbios-ssn  Samba smbd 3.X - 4.X (workgroup: WORKGROUP)
445/tcp   open  netbios-ssn  Samba smbd 3.X - 4.X (workgroup: WORKGROUP)
512/tcp   open  exec         netkit-rsh rshexec
513/tcp   open  login        OpenBSD or Solaris rlogind
514/tcp   open  tcpwrapped
1099/tcp  open  java-rmi     GNU Classpath gmicregistry
1524/tcp  open  blindshell   2-4 (RPC #100003)
2049/tcp  open  nfs          ProFTPD 1.3.1
2121/tcp  open  ftp          ProFTPD 1.3.1
3306/tcp  open  mysql        MySQL 5.0.51a-3ubuntu5
5432/tcp  open  postgresql   PostgreSQL DB 8.3.0 - 8.3.7
5900/tcp  open  vnc          VNC (protocol 3.3)
6000/tcp  open  x11          (access denied)
6667/tcp  open  irc          UnrealIRCd
8009/tcp  open  ajp13        Apache Jserv (Protocol v1.3)
8180/tcp  open  http         Apache Tomcat/Coyote JSP engine 1.1
MAC Address: 00:0C:29:FA:DD:2A (VMware)
Service Info: Hosts: metasploitable.localdomain, irc.Metasploitable.LAN; OSs: Unix, Linux; CPE: cpe:/o:linux:linux_kernel

Service detection performed. Please report any incorrect results at https://nmap.org/submit/.
Nmap done: 1 IP address (1 host up) scanned in 12.70 seconds

(kali@kali)~$
```

nmap -sV 192.168.94.135 – 24 open TCP ports with service versions identified

Port	Service	Version	Observation
21	FTP	vsftpd 2.3.4	Outdated FTP version; a known backdoored release
22	SSH	OpenSSH 4.7p1	Remote administration exposed, outdated version
23	Telnet	Linux telnetd	Unencrypted remote login — high risk
80	HTTP	Apache httpd 2.2.8	The target web server verified above, outdated version
3306	MySQL	MySQL 5.0.51a	Database service directly reachable on the network

The scan confirms the Apache web service on port 80 is running exactly as configured, and reveals that the target exposes an unusually large number of additional services, many running outdated software — useful context for prioritising which services would need hardening in a real assessment.

Full Port List

For completeness, all 24 open ports identified by the scan are recorded below.

Port	Service
21/tcp	ftp (vsftpd 2.3.4)
22/tcp	ssh (OpenSSH 4.7p1)
23/tcp	telnet
25/tcp	smtp (Postfix)
Port	Service
53/tcp	domain (ISC BIND 9.4.2)
80/tcp	http (Apache httpd 2.2.8)
111/tcp	rpcbind
139/tcp	netbios-ssn (Samba)
445/tcp	netbios-ssn (Samba)
512/tcp	exec (netkit-rsh)
513/tcp	login (rlogind)
514/tcp	tcpwrapped
1099/tcp	java-rmi

1524/tcp	bindshell
2049/tcp	nfs
2121/tcp	ftp (ProFTPD)
3306/tcp	mysql
5432/tcp	postgresql
5900/tcp	vnc
6000/tcp	X11
6667/tcp	irc (UnrealIRCd)
8009/tcp	ajp13 (Apache Jserv)
8180/tcp	http (Tomcat)
	(24 ports total)

Analysis

Comparing this list against the single service that was intentionally configured (Apache on port 80) highlights an important lesson: a machine can be running far more than what its administrator explicitly set up. On Metasploitable, this is by design for training purposes, but on a real system, an unexpectedly large list like this would immediately prompt an investigation into which services are actually needed and which should be disabled.

8. Wireshark Findings

A live packet capture was taken on Kali's network interface while browsing to the target's web server and pinging it, then filtered to isolate three core protocols.

HTTP Traffic

The screenshot shows a Wireshark capture of HTTP traffic on interface eth0. The packet list on the left shows several HTTP packets. The selected packet (No. 364) is a GET request to 192.168.94.135. The packet details pane shows the request structure: Ethernet II, Internet Protocol Version 4, Transmission Control Protocol, and Hypertext Transfer Protocol. The packet bytes pane shows the raw data in hexadecimal and ASCII.

No.	Time	Source	Destination	Protocol	Length	Info
362	32.167773941	192.168.94.137	192.168.94.135	HTTP	364	GET /success.txt?ip=4 HTTP/1.1
364	32.198818759	192.168.94.137	192.168.94.135	HTTP	430	HTTP/1.1 200 OK (text/plain)
475	38.148662054	192.168.94.137	192.168.94.135	HTTP	480	GET / HTTP/1.1
477	38.162488122	192.168.94.135	192.168.94.137	HTTP	1198	HTTP/1.1 200 OK (text/html)
479	38.718733297	192.168.94.137	192.168.94.135	HTTP	417	GET /favicon.ico HTTP/1.1
488	38.715665869	192.168.94.135	192.168.94.137	HTTP	582	HTTP/1.1 404 Not Found (text/html)

Wireshark filter: `http - GET requests to the target web server and 200 OK responses`

The capture shows a GET / request to 192.168.94.135 followed by an HTTP/1.1 200 OK response containing the web page's HTML — confirming the same web traffic that curl retrieved earlier, but now visible at the packet level.

DNS Traffic

The screenshot shows a Wireshark capture of DNS traffic on interface eth0. The packet list on the left shows several DNS packets. The selected packet (No. 286) is a standard query for 192.168.94.2. The packet details pane shows the query structure: Ethernet II, Internet Protocol Version 4, User Datagram Protocol, and Domain Name System. The packet bytes pane shows the raw data in hexadecimal and ASCII.

No.	Time	Source	Destination	Protocol	Length	Info
286	32.079473263	192.168.94.137	192.168.94.2	DNS	73	Standard query 0xe84a AAAA ipv4only.arpa
287	32.083033726	192.168.94.137	192.168.94.2	DNS	84	Standard query 0xcd57 A detectportal.firefox.com
288	32.083152380	192.168.94.137	192.168.94.2	DNS	84	Standard query 0x2252 AAAA detectportal.firefox.com
289	32.115464919	192.168.94.2	192.168.94.137	DNS	183	Standard query response 0x681c A example.org A 184.20.26.136 A 172.66.157.237
290	32.115465705	192.168.94.2	192.168.94.137	DNS	183	Standard query response 0xc67f A example.org A 184.20.26.136 A 172.66.157.237
291	32.116941455	192.168.94.2	192.168.94.137	DNS	99	Standard query response 0x617e AAAA example.org AAAA 2606:4700:900c:6c5e:b164:4e4:525:2b85
292	32.116941731	192.168.94.2	192.168.94.137	DNS	123	Standard query response 0x2909 HTTPS cloudflare-dns.com HTTPS
293	32.116941797	192.168.94.2	192.168.94.137	DNS	129	Standard query response 0xe84a AAAA ipv4only.arpa AAAA 64:ff9b::c000:aa AAAA 64:ff9b::c000:ab
294	32.116265830	192.168.94.137	192.168.94.2	DNS	84	Standard query 0xb0b9 A detectportal.firefox.com
295	32.116762726	192.168.94.2	192.168.94.137	DNS	195	Standard query response 0x4609 A ipv4only.arpa A 192.0.0.170 A 192.0.0.171
296	32.124288444	192.168.94.2	192.168.94.137	DNS	148	Standard query response 0x2252 AAAA detectportal.firefox.com CNAME mozilla.map.fastly.net AAAA 2a04:4e42:19::347
297	32.128310257	192.168.94.2	192.168.94.137	DNS	136	Standard query response 0xcd57 A detectportal.firefox.com CNAME mozilla.map.fastly.net A 151.101.193.91
306	32.151091622	192.168.94.2	192.168.94.137	DNS	153	Standard query response 0x0000 E mozilla.map.fastly.net CNAME mozilla.map.fastly.net A 151.101.193.91
307	32.284719079	192.168.94.137	192.168.94.2	DNS	84	Standard query 0x3d09 A normandy.cdn.mozilla.net
308	32.284789970	192.168.94.137	192.168.94.2	DNS	84	Standard query 0xfad1 AAAA normandy.cdn.mozilla.net
309	32.294082168	192.168.94.137	192.168.94.2	DNS	87	Standard query 0xb133 A services.addons.mozilla.org
310	32.294498843	192.168.94.137	192.168.94.2	DNS	87	Standard query 0x1434 AAAA services.addons.mozilla.org
312	32.311062313	192.168.94.2	192.168.94.137	DNS	170	Standard query response 0x3d09 A normandy.cdn.mozilla.net CNAME normandy.tombstone.experimentor.prod.webservices.mozgcp.net A 34.4
313	32.321864224	192.168.94.2	192.168.94.137	DNS	182	Standard query response 0xfad1 AAAA normandy.cdn.mozilla.net CNAME normandy.tombstone.experimentor.prod.webservices.mozgcp.net AAA
314	32.321938662	192.168.94.2	192.168.94.137	DNS	151	Standard query response 0x0133 A services.addons.mozilla.org A 151.101.129.91 A 151.101.129.91 A 151.101.193.91
316	32.323063356	192.168.94.2	192.168.94.137	DNS	199	Standard query response 0x1434 AAAA services.addons.mozilla.org AAAA 2a04:4e42:600::347 AAAA 2a04:4e42:347 AAAA 2a04:4e42:400::34
388	37.582494945	192.168.94.137	192.168.94.2	DNS	96	Standard query 0xb1b6 A classify-client.services.mozilla.com
389	37.582572575	192.168.94.137	192.168.94.2	DNS	96	Standard query 0x82bb AAAA classify-client.services.mozilla.com
390	37.591498531	192.168.94.137	192.168.94.2	DNS	76	Standard query 0x70e0 A aus5.mozilla.org

Wireshark filter: `dns - domain name resolution queries and responses`

Standard DNS queries and responses are visible for various domains resolved through the gateway (192.168.94.2), showing how a device looks up an IP address before establishing any connection.

ICMP Traffic

The image shows a Wireshark packet capture of ICMP traffic. The filter is set to 'icmp'. The packet list shows 1035 packets, with 300 displayed. The packet details pane shows the structure of an ICMP Echo request.

No.	Time	Source	Destination	Protocol	Length	Info
970	224.354652510	192.168.94.137	192.168.94.135	ICMP	98	Echo (ping) request id=0xa708, seq=139/35584, ttl=64 (reply in 971)
971	224.355779880	192.168.94.135	192.168.94.137	ICMP	98	Echo (ping) reply id=0xa708, seq=139/35584, ttl=64 (request in 970)
972	225.360616193	192.168.94.137	192.168.94.135	ICMP	98	Echo (ping) request id=0xa708, seq=140/35640, ttl=64 (reply in 973)
973	225.360952681	192.168.94.135	192.168.94.137	ICMP	98	Echo (ping) reply id=0xa708, seq=140/35640, ttl=64 (request in 972)
974	226.363128904	192.168.94.137	192.168.94.135	ICMP	98	Echo (ping) request id=0xa708, seq=141/36096, ttl=64 (reply in 975)
975	226.363398392	192.168.94.135	192.168.94.137	ICMP	98	Echo (ping) reply id=0xa708, seq=141/36096, ttl=64 (request in 974)
976	227.365002490	192.168.94.137	192.168.94.135	ICMP	98	Echo (ping) request id=0xa708, seq=142/36352, ttl=64 (reply in 977)
977	227.365317705	192.168.94.135	192.168.94.137	ICMP	98	Echo (ping) reply id=0xa708, seq=142/36352, ttl=64 (request in 976)
980	228.368424953	192.168.94.137	192.168.94.135	ICMP	98	Echo (ping) request id=0xa708, seq=143/36608, ttl=64 (reply in 981)
981	228.371830674	192.168.94.135	192.168.94.137	ICMP	98	Echo (ping) reply id=0xa708, seq=143/36608, ttl=64 (request in 980)
985	229.377216961	192.168.94.137	192.168.94.135	ICMP	98	Echo (ping) request id=0xa708, seq=144/36864, ttl=64 (reply in 986)
986	229.384843774	192.168.94.135	192.168.94.137	ICMP	98	Echo (ping) reply id=0xa708, seq=144/36864, ttl=64 (request in 985)
987	230.378643844	192.168.94.137	192.168.94.135	ICMP	98	Echo (ping) request id=0xa708, seq=145/37120, ttl=64 (reply in 988)
988	230.378916586	192.168.94.135	192.168.94.137	ICMP	98	Echo (ping) reply id=0xa708, seq=145/37120, ttl=64 (request in 987)
991	231.390827985	192.168.94.137	192.168.94.135	ICMP	98	Echo (ping) request id=0xa708, seq=146/37376, ttl=64 (reply in 992)
992	231.402465570	192.168.94.135	192.168.94.137	ICMP	98	Echo (ping) reply id=0xa708, seq=146/37376, ttl=64 (request in 991)
994	232.400835362	192.168.94.137	192.168.94.135	ICMP	98	Echo (ping) request id=0xa708, seq=147/37632, ttl=64 (reply in 995)
995	232.401138990	192.168.94.135	192.168.94.137	ICMP	98	Echo (ping) reply id=0xa708, seq=147/37632, ttl=64 (request in 994)
1003	233.413258092	192.168.94.137	192.168.94.135	ICMP	98	Echo (ping) request id=0xa708, seq=148/37888, ttl=64 (reply in 1004)
1004	233.413743127	192.168.94.135	192.168.94.137	ICMP	98	Echo (ping) reply id=0xa708, seq=148/37888, ttl=64 (request in 1003)
1013	234.436606218	192.168.94.137	192.168.94.135	ICMP	98	Echo (ping) request id=0xa708, seq=149/38144, ttl=64 (reply in 1014)
1014	234.437126232	192.168.94.135	192.168.94.137	ICMP	98	Echo (ping) reply id=0xa708, seq=149/38144, ttl=64 (request in 1013)
1031	235.460912201	192.168.94.137	192.168.94.135	ICMP	98	Echo (ping) request id=0xa708, seq=150/38400, ttl=64 (reply in 1032)
1032	235.461152678	192.168.94.135	192.168.94.137	ICMP	98	Echo (ping) reply id=0xa708, seq=150/38400, ttl=64 (request in 1031)

Frame 493: Packet, 98 bytes on wire (784 bits), 98 bytes captured (784 bits) on interface eth0, id 0
 Ethernet II, Src: VMware_05:df:85 (08:0c:29:05:df:85), Dst: VMware_fa:dd:2a (08:0c:29:fa:dd:2a)
 Internet Protocol Version 4, Src: 192.168.94.137, Dst: 192.168.94.135
 Internet Control Message Protocol

Wireshark filter: icmp – ping echo request/reply pairs between Kali and the target

Repeated ICMP echo request/reply pairs between 192.168.94.137 and 192.168.94.135 confirm basic reachability testing (ping) between the attacker and target machines on the lab network.

9. Mini Project Output – Full Workflow

Stage	Result
Network Setup	Kali Linux (192.168.94.137) and Metasploitable 2 (192.168.94.135) connected on an isolated VMware NAT lab network
Services Running	Apache HTTP Server confirmed active on the target, serving its default page and linked vulnerable web apps
Scan Results	Nmap identified 24 open TCP ports, including the target web service (Apache 2.2.8) and several other exposed services
Traffic Capture	Wireshark captured and confirmed HTTP, DNS and ICMP traffic between the two lab machines

10. Conclusion & Learning

This assignment built directly on previous weeks' networking and scanning fundamentals, this time focused specifically on validating that a running service (a web server) is both functional and visible to standard reconnaissance tools. Setting up the lab, confirming the service with curl, scanning it with Nmap, and then observing the same traffic in Wireshark reinforced how these tools complement each other: curl proves a service works, Nmap proves it is discoverable, and Wireshark proves exactly what is happening on the wire.

A key learning point was recognising that legacy training systems like Metasploitable intentionally run many outdated, exposed services — useful for practice, but a clear illustration of what NOT to expose on a production network. This reinforces the security principle, from earlier weeks, of minimising unnecessary open services and keeping software up to date.

11. Task Breakdown Summary

Task	Description	Status
Task 1	Set up a basic lab network (Kali Linux attacker + Metasploitable target) on VMware, connected on the same subnet	Completed
Task 2	Install & verify a web server (Apache) on the target machine, confirmed reachable and responding	Completed
Task 3	Scan the target with Nmap to identify open ports and running services	Completed
Task 4	Capture and filter network traffic with Wireshark (HTTP, DNS, ICMP)	Completed
Task 5	Document the complete workflow as a mini project with report and presentation	Completed

12. Challenges Faced

- Ensuring the Apache service was actually running on the target before scanning — verified directly on the machine using the init script rather than assuming the service was active.
- Filtering Wireshark captures to isolate only the relevant HTTP, DNS and ICMP traffic from the large volume of background packets generated by normal browsing and lab activity.

- Correlating the exact same web request across three different views — the curl output, the Nmap service detection, and the Wireshark packet capture — to build a single, consistent picture of the same underlying event.

13. Bonus Work Considered

As optional bonus scope, this lab's design already supports adding further virtual machines to the same NAT network for a larger topology, and the Nmap scan performed (-sV, full port range) goes beyond a minimal default scan by including service version detection. Future extensions could include analysing additional protocols (e.g., FTP or SMTP handshakes) and adding a dedicated firewall/pfSense VM between the attacker and target to observe how filtering changes the scan and capture results.

14. Conclusion

This assignment successfully demonstrated the end-to-end workflow of initial network reconnaissance and security testing within an isolated lab environment. By integrating multiple tools—using curl to confirm service responsiveness, Nmap to map out discoverable entry points, and Wireshark to analyze underlying packet exchanges—the exercise highlighted how these utilities complement one another to provide a complete picture of network activity. A key takeaway from the lab is recognizing that default or legacy systems often expose a wide array of legacy services and outdated software, reinforcing the fundamental cybersecurity principle of minimizing unnecessary open ports, reducing the attack surface, and maintaining rigorous patch management in production environments.